Project Title: Concept drift–aware cloud ML pipeline for smart inventory management

Group Number:

4

Group Members (Names and IDs):

Ali Rizvi #3802911

Fahim Faisal #3809190

Refat Ishrak Hemel #3810002

Course Details:

CS6905 Cloud Info Management Systems

Instructor:

Professor Shadi Aljendi

Date: February 26, 2026

Problem Statement

Most businesses still forecast demand using static models or manual rules, and the result is predictable, they either overstock and tie up capital, or run out of inventory and lose sales. Machine learning helps, but the way most companies deploy ML is basically "train once and forget." The model works fine at first, then slowly gets worse as customer behavior shifts with new seasons, promotions, or market changes. Nobody retrains it because the infrastructure isn't set up for that.

There's also a cost problem. Inventory forecasting doesn't need to run 24/7, most businesses only need updated predictions once a day or a week. But traditional deployments keep servers running all the time, anyway, paying for idle compute.

This project builds a serverless cloud system that handles both issues: it does periodic demand forecasting using ML, and it retrains the model automatically as new sales data comes in. We want to see whether this adaptive approach gives better predictions over time compared to a fixed model, and whether serverless makes it cheaper to run.

Proposed Solution

We're building a serverless inventory forecasting system on AWS that ingests sales data, runs ML-based demand predictions, and supports automated retraining. Sales transactions come in over time, and the system uses that data to make and improve its forecasting decisions.

We'll compare two strategies. The first trains a model once on historical data and uses it going forward without updates. The second retrains periodically as new weekly sales data comes in, so it can adapt to changing patterns. The goal is to measure whether retraining helps enough to justify the extra compute and complexity.

The whole thing is serverless, Lambda functions only run when triggered, either on a schedule or by new data arriving. No always-on servers. This makes sense for inventory forecasting since predictions only happen periodically and a few seconds of cold-start delay doesn't matter when you're planning a day or a week ahead.

NoSQL Database Use

We're using DynamoDB because the system handles data that looks very different depending on where it comes from. The pipeline ingests sales transactions, weather observations, computed features, prediction outputs, and model metadata, each with its own structure. Sales records have store ID, SKU, quantity, price. Weather data has temperature, humidity, conditions. Model metadata tracks hyperparameters, feature versions, training windows, and accuracy metrics that change shape every time we iterate. In a relational database, every new data source or feature tweak means running schema migrations, which gets old fast during iterative development.

DynamoDB avoids this because items in the same table can have completely different attributes, so all these record types coexist without a fixed schema upfront. DynamoDB Streams is the other big reason, it gives us event-driven integration with Lambda, so new records automatically kick off

feature re-computation or retraining. Without Streams we'd need separate change-data-capture infrastructure for the same behavior.

Machine Learning Component

For the baseline we use a simple moving average, windowed averages (7-day, 30-day) over past sales. No training, just a lower bound to compare against.

The main model is a weighted ensemble of Random Forest, XGBoost, and Linear Regression. No single model handles everything well on its own, the tree-based ones pick up non-linear interactions (promotions with weather, seasonality effects) while Linear Regression is more stable for general trends. Weighted averaging gives better predictions than any one alone. All three train fast and produce small files, which matters for retraining inside Lambda's limits. Features include temporal signals (day of week, month, holidays), units sold, units ordered, inventory level, external context (weather, season, promotions), and price features (discounts, competitor gaps).

We simulate concept drift by splitting the data chronologically into weekly or biweekly windows. The static strategy trains on the first window and never updates. The adaptive strategy retrains at each boundary using recent data. This shows how a model trained on winter patterns lose accuracy once spring kicks in, while the retrained version adapts. We track MAE and RMSE per window plus retraining duration and Lambda cost.

AWS Tools

Everything runs serverless. Here's what and why:

DynamoDB: Main database for sales, weather, features, predictions, and model metadata. Schema-flexible, and Streams triggers Lambdas when new records arrive for event-driven processing.

Lambda: Compute, ingestion, feature engineering, inference (model from S3, predictions back to DynamoDB), and retraining. Used instead of EC2 because forecasting is periodic, no point paying for idle servers.

S3: Model artifacts, dataset, and intermediate data. Versioning keeps old models for rollback.

EventBridge: Schedules weekly retraining and chains steps like post-retraining validation.

API Gateway: REST endpoints for forecast requests and inventory queries via Lambda.

CloudWatch: Logging, execution metrics, custom metrics like per-cycle MAE, and alarms for failures or accuracy drops.

Amplify: Hosts the frontend dashboard for visualizing forecasts, inventory state, and retraining metrics